Department of Artificial Intelligence and Machine Learning
B.Tech. Sem: V Subject: Full Stack Development Using NextJs (DJS23AMD302L)
EXPERIMENT NO. 8

AIM:

To build strict multi-role permission layers (Admin / User / Guest) utilizing Next.js 16 Edge runtime middleware (proxy.ts / middleware.ts) validation to block unauthorized server data access and control protected subtrees.

SOFTWARE REQUIRED:

- **Operating System:** Linux / macOS / Windows 10/11
- **Node.js runtime environment:** (v18.17.0 or higher)
- **Package Manager:** npm or pnpm
- **Code Editor:** Visual Studio Code

THEORY:

- **Edge Middleware Architecture:** Next.js middleware executes before a request completes, running directly at the network boundary before route handlers, layouts, or pages evaluate. This enables request inspection, rewrite dispatching, header mutation, and early HTTP 302/403 status interceptions without invoking heavy downstream Node.js server execution contexts.
- **Role-Based Access Control (RBAC):** RBAC enforces user authorization based on pre-assigned roles within an organization or system. In this hierarchy:
  - **Guest:** Unauthenticated entity permitted access only to public index, informational pages, or login endpoints.
  - **User:** Authenticated entity allowed access to standard profile dashboards and personal data entries (/dashboard/*).
  - **Admin:** Privileged operator with access to restricted operational controls, audit logs, and administrative configuration suites (/admin/*).
- **Cryptographic Token Verification in Edge Runtimes:** Edge runtime environments operate under lightweight V8 worker isolates that do not include standard Node.js native binary libraries (e.g., standard Node crypto or C++ bindings). Stateless role verification relies on lightweight Web Cryptography APIs (or libraries like jose) to decode and verify HMAC-SHA256 signed JSON Web Tokens (JWT) directly inside the middleware pipeline.
- **Route Matcher Optimization:** Next.js provides declarative matcher regex filters that constrain middleware execution to specific route segments. Static assets, Next.js internal bundles (/_next/*), and image assets are explicitly bypassed to avoid performance bottlenecks.

STEPS / ALGORITHM:

Step 1: Install Token Handling Dependencies

Install jose for Edge-compatible JWT encoding and verification:

```
npm install jose
```

Add your HMAC secret key to .env:

```
JWT_SECRET="super-secure-multi-role-secret-key-32-chars-min"
```

Step 2: Create Token Utility & RBAC Definitions (app/lib/auth-token.ts)

Define user roles and symmetric token signing/verification helpers using Web Crypto standards:

```
import { SignJWT, jwtVerify } from 'jose';

export type UserRole = 'ADMIN' | 'USER' | 'GUEST';

export interface TokenPayload {
  userId: string;
  email: string;
  role: UserRole;
}

const SECRET = new TextEncoder().encode(
  process.env.JWT_SECRET || 'super-secure-multi-role-secret-
key-32-chars-min'
);

// Sign a session token containing the specific role
export async function signRoleToken(payload: TokenPayload):
Promise<string> {
  return await new SignJWT({ ...payload })
    .setProtectedHeader({ alg: 'HS256' })
    .setIssuedAt()
    .setExpirationTime('2h')
    .sign(SECRET);
}

// Verify and extract role claims inside Edge runtime
export async function verifyRoleToken(token: string):
Promise<TokenPayload | null> {
```

```
  try {
    const { payload } = await jwtVerify(SECRET, token);
    return payload as unknown as TokenPayload;
  } catch {
    return null;
  }
}
```

Step 3: Implement Edge Proxy Middleware (proxy.ts / middleware.ts)

Create the middleware in your project root or src/ folder to intercept requests, enforce role boundaries, and redirect unauthorized attempts:

```
import { NextResponse } from 'next/server';
import type { NextRequest } from 'next/server';
import { verifyRoleToken } from './app/lib/auth-token';

export async function middleware(request: NextRequest) {
  const { pathname } = request.nextUrl;
  const token =
request.cookies.get('session_role_token')?.value;

  // Verify token payload if present
  const session = token ? await verifyRoleToken(token) :
null;
  const userRole = session ? session.role : 'GUEST';

  // Define access permission boundaries
  const isAdminRoute = pathname.startsWith('/admin');
  const isUserRoute = pathname.startsWith('/dashboard');
  const isAuthRoute = pathname.startsWith('/login');

  // Rule 1: Block unauthenticated users (Guests) from
private routes
  if ((isAdminRoute || isUserRoute) && userRole === 'GUEST')
{
    const loginUrl = new URL('/login', request.url);
    loginUrl.searchParams.set('redirect', pathname);
    return NextResponse.redirect(loginUrl);
  }

  // Rule 2: Strict Admin Route Enforcement
  if (isAdminRoute && userRole !== 'ADMIN') {
    // Insufficient permissions: redirect to standard
dashboard or unauthorized page
```

```
    return NextResponse.redirect(new URL('/unauthorized',
request.url));
  }

  // Rule 3: Forward user details to downstream Server
Components via headers
  const requestHeaders = new Headers(request.headers);
  requestHeaders.set('x-user-role', userRole);
  if (session) {
    requestHeaders.set('x-user-id', session.userId);
  }

  return NextResponse.next({
    request: {
      headers: requestHeaders,
    },
  });
}

// Configure route matching filters
export const config = {
  matcher: ['/admin/:path*', '/dashboard/:path*', '/login'],
};
```

Step 4: Implement Role Simulation Server Actions (app/actions/roleAuthActions.ts)

Create actions that set signed role cookies to simulate different permission tiers during testing:

```
'use server';

import { cookies } from 'next/headers';
import { redirect } from 'next/navigation';
import { signRoleToken, UserRole } from '@/app/lib/auth-
token';

export async function switchRoleAction(role: UserRole) {
  const cookieStore = await cookies();

  if (role === 'GUEST') {
    cookieStore.delete('session_role_token');
    redirect('/login');
  }
```

```
  const token = await signRoleToken({
    userId: `usr_${role.toLowerCase()}_123`,
    email: `${role.toLowerCase()}@example.com`,
    role,
  });

  cookieStore.set('session_role_token', token, {
    httpOnly: true,
    secure: process.env.NODE_ENV === 'production',
    sameSite: 'lax',
    path: '/',
    maxAge: 7200,
  });

  if (role === 'ADMIN') {
    redirect('/admin');
  } else {
    redirect('/dashboard');
  }
}
```

Step 5: Construct Role Switcher & Protected UI Views

A. Role Control Center (app/login/page.tsx)

```
import { switchRoleAction } from
'@/app/actions/roleAuthActions';

export default function LoginPage() {
  return (
    <div style={{ maxWidth: '500px', margin: '40px auto',
padding: '24px', fontFamily: 'sans-serif', border: '1px solid
#e2e8f0', borderRadius: '12px' }}>
      <h2>Role-Based Access Control Simulator</h2>
      <p style={{ color: '#64748b', fontSize: '14px'
}}>Select an identity below to issue a signed role token and
test middleware guards.</p>

      <div style={{ display: 'flex', flexDirection: 'column',
gap: '12px', marginTop: '20px' }}>
        <form action={switchRoleAction.bind(null, 'ADMIN')}>
          <button type="submit" style={{ width: '100%',
```

```
padding: '10px', background: '#dc2626', color: '#fff',
border: 'none', borderRadius: '6px', cursor: 'pointer' }}>
            Assume ADMIN Role (Access: /admin & /dashboard)
          </button>
        </form>

        <form action={switchRoleAction.bind(null, 'USER')}>
          <button type="submit" style={{ width: '100%',
padding: '10px', background: '#2563eb', color: '#fff',
border: 'none', borderRadius: '6px', cursor: 'pointer' }}>
            Assume USER Role (Access: /dashboard only)
          </button>
        </form>

        <form action={switchRoleAction.bind(null, 'GUEST')}>
          <button type="submit" style={{ width: '100%',
padding: '10px', background: '#64748b', color: '#fff',
border: 'none', borderRadius: '6px', cursor: 'pointer' }}>
            Assume GUEST Role (Clear Cookies / Public only)
          </button>
        </form>
      </div>
    </div>
  );
}
```

B. Admin Protected Route (app/admin/page.tsx)

```
import { headers } from 'next/headers';
import Link from 'next/link';

export default async function AdminPage() {
  const headerList = await headers();
  const currentRole = headerList.get('x-user-role');

  return (
    <div style={{ maxWidth: '700px', margin: '40px auto',
padding: '24px', fontFamily: 'sans-serif', border: '2px solid
#ef4444', borderRadius: '12px', background: '#fef2f2' }}>
      <h1 style={{ color: '#991b1b' }}>Admin Clearance
Terminal</h1>
      <p>Middleware validation check passed: Verified Role =
<strong>{currentRole}</strong></p>
```

```
      <div style={{ marginTop: '20px', display: 'flex', gap:
'15px' }}>
        <Link href="/dashboard" style={{ color: '#2563eb'
}}>Go to User Dashboard</Link>
        <Link href="/login" style={{ color: '#2563eb'
}}>Switch Role</Link>
      </div>
    </div>
  );
}
```

C. User Protected Route (app/dashboard/page.tsx)

```
import { headers } from 'next/headers';
import Link from 'next/link';

export default async function DashboardPage() {
  const headerList = await headers();
  const currentRole = headerList.get('x-user-role');

  return (
    <div style={{ maxWidth: '700px', margin: '40px auto',
padding: '24px', fontFamily: 'sans-serif', border: '1px solid
#cbd5e1', borderRadius: '12px', background: '#f8fafc' }}>
      <h1 style={{ color: '#0f172a' }}>Standard User
Dashboard</h1>
      <p>Access Granted: Current Verified Role =
<strong>{currentRole}</strong></p>
      <div style={{ marginTop: '20px', display: 'flex', gap:
'15px' }}>
        <Link href="/admin" style={{ color: '#dc2626'
}}>Attempt to access /admin</Link>
        <Link href="/login" style={{ color: '#2563eb'
}}>Switch Role</Link>
      </div>
    </div>
  );
}
```

D. Unauthorized Interception Page (app/unauthorized/page.tsx)

```
import Link from 'next/link';

export default function UnauthorizedPage() {
  return (
    <div style={{ maxWidth: '500px', margin: '60px auto',
textAlign: 'center', fontFamily: 'sans-serif' }}>
      <h1 style={{ color: '#dc2626', fontSize: '48px',
margin: 0 }}>403</h1>
      <h2>Access Denied: Insufficient Clearance</h2>
      <p style={{ color: '#64748b' }}>Middleware intercepted
your request. You do not possess the required
<strong>ADMIN</strong> permissions to view this resource.</p>
      <Link href="/login" style={{ display: 'inline-block',
marginTop: '15px', padding: '8px 16px', background:
'#0f172a', color: '#fff', borderRadius: '6px',
textDecoration: 'none' }}>
        Return to Role Selector
      </Link>
    </div>
  );
}
```

Step 6: Validate Multi-Role Guards

1. Start the development server:
   npm run dev

2. Navigate directly to http://localhost:3000/admin. Verify that as a **Guest**, the middleware intercepts the navigation and redirects to /login?redirect=/admin.
3. In /login, select **Assume USER Role**. Navigate to http://localhost:3000/dashboard (granted).
4. From the User dashboard, click the link to access /admin. Verify the middleware blocks access and redirects to /unauthorized (403).
5. Return to /login and select **Assume ADMIN Role**. Navigate to both /dashboard and /admin to verify full clearance.

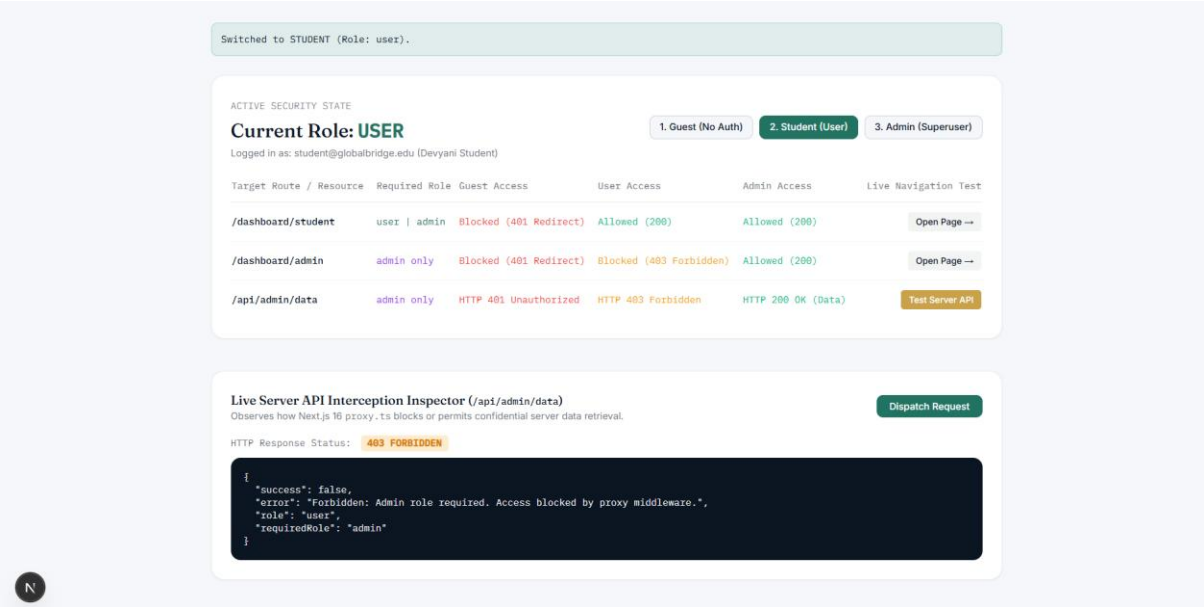
OBSERVATION:

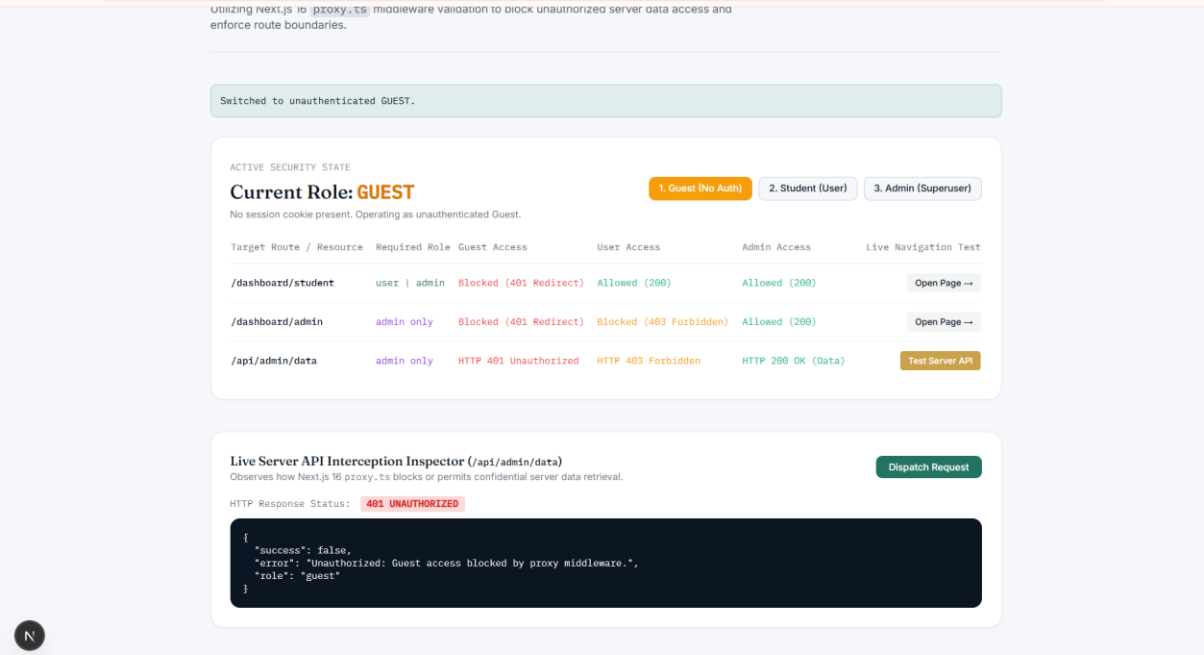
Observe how middleware executes at the Edge before route rendering begins. When a user with the USER role attempts to hit /admin, the request never triggers Server Component compilation for app/admin/page.tsx.

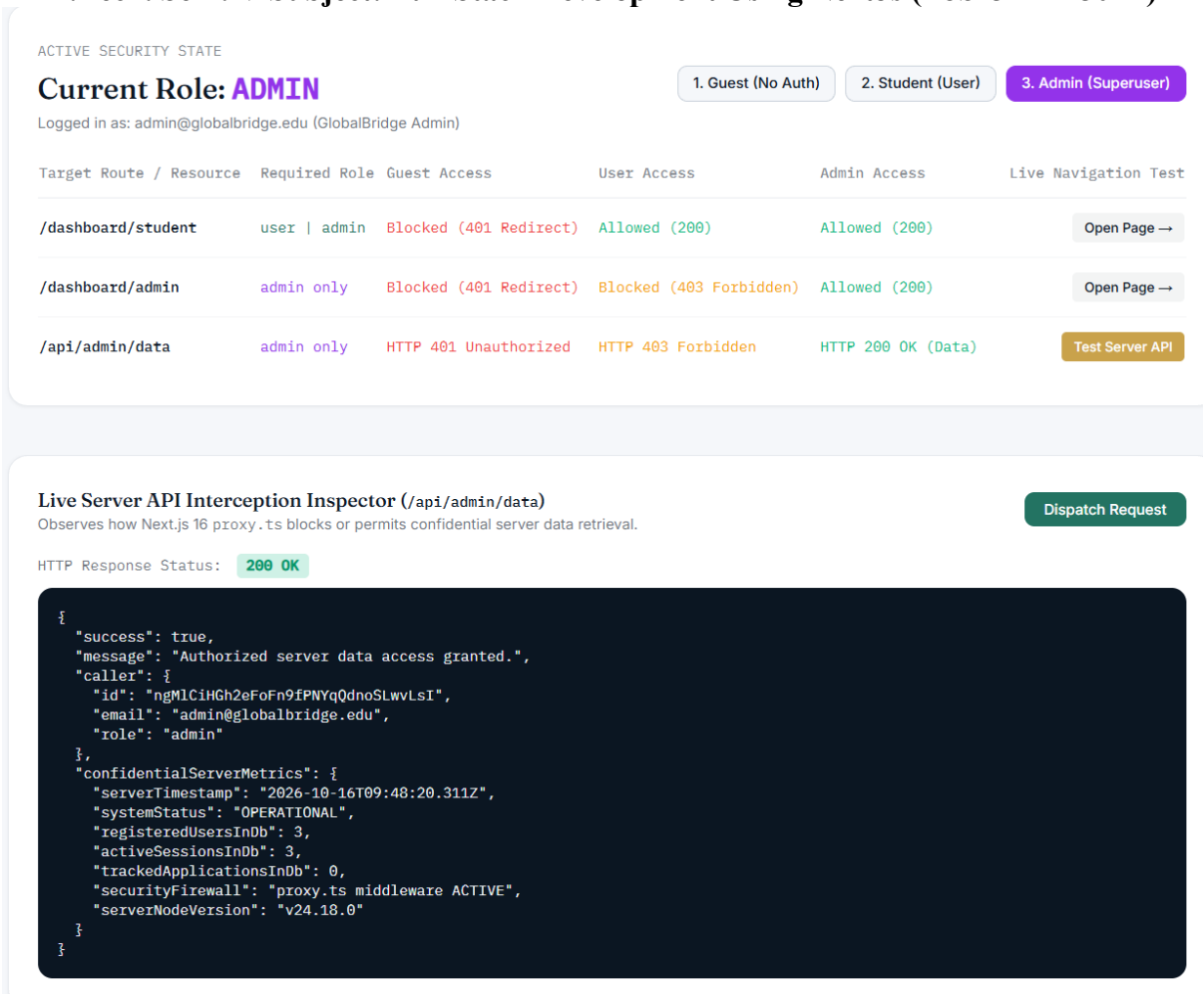
Instead, the middleware halts evaluation at the proxy layer, checks the verified token payload, and redirects the client, conserving server compute and preventing sensitive data leakage.

CONCLUSION:

Strict multi-role permission layers (Admin/User/Guest) were successfully implemented using Next.js Edge proxy middleware validation. The experiment verified Edge-compatible cryptographic JWT validation, route guard enforcement, and request header propagation across protected subtrees.







POST LAB QUESTION:

Q: Why should sensitive access control not rely solely on layout-level checks (layout.tsx), and why is Edge middleware considered a superior perimeter security barrier?

A: Layout-level security checks execute only after the Next.js server resolves the route, initializes the React component tree, and often begins resolving nested server components and data fetching queries. If an authorization check inside a layout fails, downstream components or sibling parallel routes may have already executed sensitive backend queries. Edge middleware runs ahead of the routing and component pipeline at the network boundary. Unauthorized requests are intercepted and redirected before any downstream code, database connections, or server components are instantiated, preventing side-channel data leaks and reducing unnecessary server compute overhead.